

Subject: Adv. Java

Date: 12/9/24

• Section- A (5*2=10)

1. Explain the distinctions among the execute, executeQuery, and executeUpdate methods.

Ans (i) executeQuery():

• Used for Select statement.

• Return a ResultSet containing the data retrieved from the DB.

(ii) executeUpdate():

• Used for DML (Insert, Update, Delete) statement.

• Returns an int representing the number of rows affected.

(iii) execute():

• Used for any SQL statement.

• Return a boolean value: 'True' if the result is a ResultSet (for select), 'False' if it's an update count.

2. What is JDBC? How to you connect to a db using JDBC.

Ans → JDBC (Java Database Connectivity) is an API that allows Java applications to connect and interact with database. It provides methods to execute SQL queries and manage database connections.

Step TO connect to a database using JDBC -

(i) Load the driver:

```
Class.forName("com.mysql.jdbc.Driver");
```

(ii) Establish a connection:

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/11db-mysql", "username", "password");
```

(iii) Create a statement:

```
Statement stmt = con.createStatement();
```

(iv) Execute Queries:

```
ResultSet rs = stmt.executeQuery("select * from table-name");
```

(v) Close the connection:

```
con.close();
```

These are five steps which through we can connect a database using JDBC.

Subject: Subject:

Q3. What are the advantages of Prepared Statement over Statement?

Ans. Advantages of Prepared Statement over Statement:

- (i) Prevents SQL injection.
- (ii) Faster Execution (Precompiled and cached queries).
- (iii) Dynamic Parameters for flexibility.
- (iv) Cleaner and more readable code.

4. What are the JDBC API Components?

Ans. JDBC API Components are -

- (i) DriverManager
- (ii) Connection
- (iii) Statement
- (iv) ResultSet
- (v) SQLException
- (vi) Data Source

Q5. What is Driver Manager in JDBC.

Ans. DriverManager in JDBC is a class that manages database drivers and establishes connections to the database. It selects the appropriate driver for a given database URL and returns a 'Connection' object that allows interaction with the database.

ex = `Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/db-name", "username", "password");`

• Section - B (3x7 = 21)

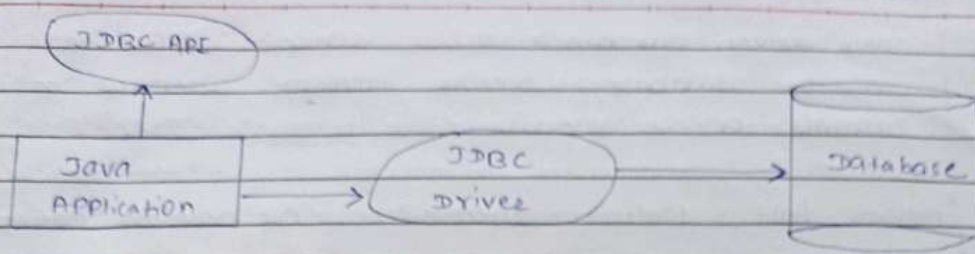
1. What do you mean by JDBC Driver? Explain the type of driver in details.

Ans. JDBC Driver is a software component that enables Java application to interact with the database.

Subject: _____

22BCN11068

Date: / / 63



There are 4 types of JDBC drivers.

(i) JDBC - ODBC bridge driver :-

- The JDBC - ODBC bridge driver uses ODBC driver to connect to the database.
- The JDBC - ODBC bridge driver converts JDBC method calls into the ODBC function calls.

Advantages :-

- Easy to use
- Can be easily connected to any database.

Disadv. - The ODBC driver need to be installed on the client machine.

(ii) Native - API driver :- The Native API driver uses the client-side libraries of the database.

- The driver converts JDBC method calls into native calls of the database API.

Advantage :- Performance upgraded than JDBC-ODBC bridge driver.

Disadv. - The Native driver needs to be installed on the each client machine.

(iii) Network Protocol Driver :- The network protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in Java.

Advantage :- Database Independent, Platform independent, Suitable for - multi-database applications.

Disadv. - Requires middleware setup, adds network overhead.

Subject:

(iv) Thin Driver (Pure Java Driver) - Converts JDBC calls directly into the database-specific protocol without native libraries or middleware.

* Adv - Fast, Platform-independent, no need for native libraries, directly communicates with the DB.

* Disadvantage - Driver depend on the Database.

2. Explain the usage of the getter and setter methods in ResultSet.

Ans. In JDBC, The 'ResultSet' interface provides methods to retrieve and update data in a database result set. The getter and setter methods are used to read and write the data from the result set.

(i) Getter Methods:- Getter methods in ResultSet are used to retrieve data from the result set. We can retrieve data based on the column index or the column label.

Methods -

- `Public int getInt(int columnIndex)`: is used to return the data of specified column index of the current row as int.
- `Public int getInt(String columnName)`: is used to retrieve a data of specified column name of the current row as int.
- `Public String getString(int columnIndex)`: is used to return the data of specified column index of the current row as String.
- `Public String getString(String columnName)`: - is used to return the data of specified column name of the current row as String.

Subject: _____

```
ex ResultSet rs = stmt.executeQuery("select id, name from emp");  
while (rs.next()) {  
    int id = rs.getInt("Id");  
    String name = rs.getString("Name");  
    S.O.P("ID: " + id + ", Name: " + name);  
}
```

(iii) Setter (Update) Methods :- Setter (update) methods are used to update the data in a ResultSet. These methods are typically used when you want to update the result set that is Updatable.

Methods :-

- UpdateInt (int columnIndex, int value) : Updates the int value in the specified column.
- UpdateString (int columnIndex, String value) : Updates the String value in the specified column.
- UpdateDouble (int columnIndex, double value) : - Updates the double value in the specified column.
- public void setInt (int paramIndex, int value) - set the integer value to the given parameter index.

```
ex Result set rs = stmt.executeQuery("select id, name from emp  
where id = 101");
```

```
if (rs.next()) {  
    rs.updateString("name", new name);  
    rs.updateDouble("salary", 60000);  
    rs.updateRow();  
}
```


Subject:

Q3 why are we utilizing Database Metadata, and what do you mean by it? Explain with example.

Ans Database metadata is an interface in java's JDBC API that provides comprehensive information about the database and its structure. It allows us to obtain metadata about the database, such as the database name, version, supported features, tables, columns, etc.

(or we Database Metadata to:

- (i) Explore database structure - Retrieve information like available tables, columns, primary keys, etc.
- (ii) Check database capabilities - Determine the features supported by the database.
- (iii) Understand database version - Fetch the database product name, version, and driver details.

ex.

```
import java.sql.*;  
class Dbmd {  
    public static void main (String args[]) {  
        try {  
            Class.forName (" oracle.jdbc.driver.OracleDriver");
```

```
            Connection con = DriverManager.getConnection ("jdbc:oracle:thin @  
                localhost:1521:xe", "root", "1234");
```

```
            DatabaseMetaData dbmd = con.getMetaData();
```

```
            System.out.println ("Driver Name: " + dbmd.getDriverName());
```

```
            System.out.println ("Driver Version: " + dbmd.getDriverVersion());
```

```
            System.out.println ("User Name: " + dbmd.getUserName());
```

```
            System.out.println ("Database Product Name: " + dbmd.getDatabaseProductVersion());
```


Subject: _____

```
System.out.println("Database Product version: " + dbmd.getDatabaseProductVersion());
```

```
conn.close();
```

```
}
```

```
catch (Exception e)
```

```
{
```

```
System.out.println(e);
```

```
}
```

```
}
```

O/P: Driver Name: Oracle JDBC Driver

Driver Version: 10.2.0.1.0.x64

Database Product Name: Oracle

Database Product Version: Oracle Database 10g

Express Edition

Release 10.2.0.1.0 - production

• Common methods in DatabaseMetadata:

- `getDatabaseProductName()`: - Gets the name of the Database.
- `getDatabaseProductVersion()`: - Gets the version of the database.
- `getDriverName()`: - Gets the name of the JDBC driver.
- `getTables()`: - Retrieves a list of tables.
- `getColumns()`: - Retrieves a list of columns.

• Section-C ($3 \times 11 = 33$)

Q. WAP of connecting Java application with MySQL database and display records using Statement interface.

Ans Sure here is the program of Statement interface that connects the Java application with MySQL database and also displays the result.

Subject: _____

```
import java.sql.*;
public class stmtInitiator {
    public static void main (String args[]) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/section1", "root", "1234");

            Statement stmt = con.createStatement();
            int insertResult = stmt.executeUpdate("insert into student
            values (1068, 'Ankit', 22)");
            System.out.println(insertResult + " record(s) inserted.");
            int updateResult = stmt.executeUpdate("update student set
            s_name = 'Vikash', age = 26 where Roll-no = 1068");
            System.out.println(updateResult + " record(s) updated.");

            int deleteResult = stmt.executeUpdate("delete from student where
            Roll-no = 1068");

            System.out.println(deleteResult + " record(s) deleted.");
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

O/P: run:

1 record(s) inserted.

1 record(s) updated.

1 record(s) deleted.

Subject: _____

2. Explain JDBC Batch Processing with example.

Ans: JDBC Batch Processing is a technique used to execute multiple SQL statements in a single batch, improving performance by reducing the number of round-trips between the application and the database. This is especially useful when you need to execute a large number of similar SQL statements, such as insert, update, or delete.

• Advantages of Batch processing - Fast performance

• Steps of batch processing -

- (i) Load the driver class
- (ii) Create Connection
- (iii) Create Statement
- (iv) Add Query in the batch
- (v) Execute Batch
- (vi) Close Connection

Example of batch processing using PreparedStatement.

#code -

```
import java.sql.*;
import java.io.*;

class BP {
    public static void main(String args[]) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/section1", "root", "1234");
            PreparedStatement ps = con.prepareStatement("insert into student values(?, ?, ?)");
```


Subject: _____

Subject: _____

```
BufferedReader br = new BufferedReader(new InputStreamReader(  
    (System.in));
```

```
while (true) {  
    System.out.println("Enter id");  
    String s1 = br.readLine();  
    int id = Integer.parseInt(s1);
```

```
    System.out.println("Enter Name");  
    String name = br.readLine();
```

```
    System.out.println("Enter age");  
    String s3 = br.readLine();  
    int age = Integer.parseInt(s3);
```

```
    ps.setInt(1, id);  
    ps.setString(2, name);  
    ps.setInt(3, salary);
```

```
    ps.addBatch();  
    S.O.P("Want to add more records y/n");  
    String ans = br.readLine();  
    if (ans.equals("n")) {  
        break;
```

```
    }  
}
```

```
ps.executeBatch();  
S.O.P("Record successfully saved");
```

```
con.close();  
}
```


Subject: _____

Date: ___/___/___ 11

```
catch (Exception e)
{
    s.o.p (e);
}
}
```

O/P: Enter id : 1029
Enter name : Ankit
Enter age : 20
Want to add more records Y/n
n
Records successfully saved.

3. WAP to executing Parametrized query and taking input from user.

Ans

```
import java.sql.*;
import java.util.Scanner;

public class ParamQueryEx {
    public static void main (String args []) {

        Scanner sc = new Scanner (System.in);
        try {
```

```
            Class.forName ("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection ("jdbc:mysql://localhost:3306/section1", "root", "4821");
```

```
            PreparedStatement stmt = con.prepareStatement ("insert into student values (?, ?, ?)");
```

```
            System.out.print ("Enter Student ID: ");
            int id = sc.nextInt();
```

```
            sc.nextLine();
```


Subject:

Subject:

```
System.out.print("Enter Student Name:");  
String name = sc.nextLine();  
  
System.out.print("Enter Student Age:");  
int age = sc.nextInt();  
  
stmt.setInt(1, id);  
stmt.setString(2, name);  
stmt.setInt(3, age);  
  
int a = stmt.executeUpdate();  
System.out.println(a + "record(s) affected.");  
  
con.close();  
scanner.close();  
}  
catch (Exception e)  
{  
    System.out.print(e);  
}  
}
```

O/P:

run:

Enter Student ID: 1025

Enter Student Name: Sumit kumar

Enter Student Age: 21

1 record(s) affected.

A+

14/9/24

the end